

CPSC-5616: Assignment 3 Report

Group Number: 9

Group Member: 3

NAME	STUDENT#	EMAIL
Anil Jagdishbhai Chavda	0445602	achavda@laurentian.ca
Shivang Darji	0442826	sdarji@laurentian.ca
Rahi Chandreshbhai Patel	0445603	rpatel20@laurentian.ca

1. Abstract

This project examines how well Random Forest and Support Vector Machines (SVM), two well-known machine learning methods, perform in a classification job using a dataset that has three classes and several characteristics. The Random Forest analysis tests several configurations by varying the maximum depth and number of trees. The best results are obtained when 100 trees are used at a depth of 15, resulting in an astounding accuracy of 96.52%. Measures like recall, accuracy, precision, and F1-score shed light on the subtle features of Random Forests' performance. Visual aids provide a full perspective of expected versus real class labels, such as confusion matrices and classification reports.

Regarding SVM, the Python scikit-learn module makes it easier to implement a supervised learning model that has been trained on a labeled dataset. The goal of this SVM model is to identify the best hyperplane for categorizing different data points in the feature space. By methodically examining different combinations of parameters, such as the gamma parameter, kernel type, and regularization parameter C, hyperparameter tuning—which is carried out by GridSearchCV—fine-tunes the model. The highest cross-validated accuracy is used to lead the selection of the best-performing SVM model, which is then rigorously tested on a different test dataset using metrics including confusion matrix, precision, recall, and F1-score to provide a thorough evaluation of its classification abilities.

As a whole, this project highlights the effectiveness of SVM and Random Forest in classification problems, emphasizing the significance of careful setup in Random Forest and the influence of optimizing hyperparameters to improve SVM model performance. The results suggest that, in order to obtain the best results in classification tasks, a nuanced approach to algorithm selection and parameter optimization is necessary.

2. Steps of this Project

a) Data Preprocessing:

To prepare the data for machine learning tasks, a number of data pretreatment steps were carried out in this research. The following is a quick rundown of the data preparation procedures used:

- i. Resampling: One class had noticeably more samples than the others in the initial dataset due to an unbalanced distribution of classes. Resampling was done to balance the classes in order to solve this problem. This entailed undersampling the majority class and arbitrarily oversampling the minority classes.
- ii. Feature extraction: Different sensor readings were represented by several columns in the dataset. The first ten columns were chosen as features, containing orientation quaternion values and accelerometer readings, in order to extract pertinent characteristics from these columns.
- iii. Standardization: It's critical to normalize the characteristics to have zero mean and unit variance before supplying them to machine learning algorithms. The StandardScaler from the sklearn.preprocessing module was used for this. Because standardization guarantees that all characteristics have a same scale, it helps machine learning algorithms perform better.
- iv. Train-Test Split: The train_test_split function from the sklearn.model_selection module was used to divide the dataset into training and testing sets. To assess how well the trained models function on untested data, this phase is essential. In this project, a test size of 30% was used, meaning that 70% of the data was used for training and 30% for testing.

These preprocessing steps were taken to ensure the data is in a format suitable for training and evaluating machine learning models. The data was ready for additional analysis and model training by balancing the classes, extracting pertinent features, standardizing the data, and dividing it into training and testing sets.

b) Feature Computation:

In order to produce meaningful features that can be applied to classification, the feature computation in this project entails taking pertinent information out of the input data. The type of data being used and the issue at hand determine which specific features are computed.

It appears from the information given that the project uses Random Forests and Support Vector Machines (SVM) for classification. The Radial Basis Function (RBF) kernel is probably used to compute the features for SVM, and various values of the kernel coefficient (gamma) and regularization parameter (C) are tested. The computed features are used to train the SVM model, which is then assessed via cross-validation. The accuracy score is then reported.

The entropy or Gini impurity is probably used as the splitting criterion when computing the features for Random Forests. To find the optimal combination, the number of Random Forests (estimators) and the maximum depth of each tree are varied. The Random Forest model is assessed through cross-validation after being trained on the computed features.

The results include a report on the classification report, confusion matrix, and accuracy score.

It is noteworthy that providing a more thorough explanation of the feature computation is challenging in the absence of additional details regarding the particular dataset and issue. But generally speaking, feature computation entails converting the unprocessed data into a format that machine learning algorithms can use to accurately predict or classify data.

c) Data Augmentation:

Data augmentation is used in this project to expand the training dataset's size and variety. This enhances the machine learning models' functionality and generalizability.

Resampling and windowing are two methods of data augmentation that were applied in this project.

The process of generating fixed-size windows from the original time series data is called resampling. In order to ensure that each window has adequate data for feature extraction, this is done. The window size, defined as a parameter, determines how many data points are included in each window. By resampling the data into windows, we can extract features from each window and use them as inputs for the machine learning models.

Sliding a window over time series data with a certain overlap is called windowing. This enables us to extract various windows from a single time series by capturing distinct data segments. We can produce more training samples by changing the windows' starting positions and levels of overlap. This enhances the dataset's diversity and strengthens the model's capacity to generalize to previously undiscovered data.

Overall, by offering a bigger and more varied training dataset, data augmentation techniques like resampling and windowing contribute to improving the performance and resilience of machine learning models.

d) Model Selection:

The GridSearchCV function from the `sklearn.model_selection` module is used in this project to select models. GridSearchCV is a hyperparameter tuning technique that looks for the optimal set of hyperparameters for a particular machine learning model.

The parameter grid is defined as a dictionary, where a list of potential values to be tested corresponds to each key, which represents a hyperparameter. Here, the regularization parameter ('C'), the kernel coefficient ('gamma'), and the kernel type ('kernel') are the hyperparameters that are being adjusted.

Next, an object called GridSearchCV is created, and the Support Vector Classification (SVC) model is used as the estimator. When the `refit` parameter is set to `True`, the model will be re-fitted using all of the training set using the optimal parameters that were

discovered during the grid search. With the verbose parameter set to 3, thorough logging is provided while the grid search is being conducted.

Next, the training set is used to fit the model and the grid search is executed. During the grid search, the model is fitted multiple times with different hyperparameter combinations. Cross-validation is used to evaluate how well each combination works. The number of folds for cross-validation is set to 5.

The grid search yields the optimal set of hyperparameters based on the highest cross-validation score. The best model is then used to make predictions on the test set.

In general, the process of selecting a model for this project entails using GridSearchCV to systematically search for the optimal set of hyperparameters and using cross-validation to assess each combination's performance. This aids in determining the model's ideal hyperparameters, improving performance and generalization.

e) Performance Analysis:

In machine learning, performance analysis entails assessing a model's performance using a variety of evaluation metrics, including confusion matrix, accuracy, and F1-score. These metrics give information about the model's performance and aid in identifying its advantages and disadvantages.

1. **Accuracy:** This popular measure evaluates the overall accuracy of the model's predictions. Divide the total number of predictions by the number of accurate predictions to perform the computation. However, accuracy alone might not provide a complete picture of the model's performance, especially when dealing with imbalanced datasets.
2. **F1-score:** This metric offers a fair assessment of the model's performance by combining recall and precision. It can be helpful when there is a class imbalance in the dataset because it is the harmonic mean of precision and recall. The F1-score has a range of 0 to 1, where 1 represents the highest possible score.
3. **Confusion Matrix:** A table summarizing a classification model's performance is called a confusion matrix. By displaying the quantity of true positives, true negatives, false positives, and false negatives, it offers a thorough analysis of the model's predictions. Numerous metrics, including precision, recall, and F1-score, can be computed from the confusion matrix.
 - True Positive (TP): The model correctly predicted the positive class.
 - True Negative (TN): The model correctly predicted the negative class.
 - False Positive (FP): The model incorrectly predicted the positive class.
 - False Negative (FN): The model incorrectly predicted the negative class.
4. **Precision:** The ratio of true positives to the total of false positives and true positives is known as precision. It assesses how well the model can recognize positive examples. A minimal false positive rate is indicated by a high precision.
5. **Recall:** The ratio of true positives to the total of true positives and false negatives is called recall, which is also referred to as sensitivity or true positive rate. It assesses how

- well the model can recognize every positive instance. A low false negative rate is indicated by a high recall.
6. **Macro Average:** Macro average assigns equal weight to each class by calculating the average performance across all classes. When all classes are equally important or the dataset is balanced, it is helpful.
 7. **Weighted Average:** This method determines the average performance of each class and gives classes with more instances a higher weight. When there is an imbalance in the dataset or when some classes are more significant than others, it may be helpful.
- These evaluation metrics help gauge a model's effectiveness and can guide further improvements or changes to the model's parameters or features. To fully understand the model's performance, it is imperative to consider a range of metrics.

Strengths and Weaknesses of the chosen ML algorithms

Random Forest:

Strengths:

1. **Robustness:** Compared to other algorithms, Random Forest is less prone to overfitting. It increases the model's capacity for generalization and lowers variance by combining several Random Forests.
2. **Feature Importance:** Random Forest offers a feature importance metric that makes it easier to comprehend how each feature contributes to the classification task.
3. **Managing Missing Values:** Random Forest can manage the dataset's missing values without the need for imputation. To deal with missing values during the tree-building process, it employs surrogate splits.
4. **Outlier Robustness:** Random Forest exhibits robustness against dataset outliers. The impact of outliers on the final predictions is lessened by averaging several Random Forests.

Weaknesses:

1. **Interpretability:** Because Random Forest models are ensemble in nature, they are difficult to interpret. Understanding the model's decision-making process can be difficult.
2. **Computationally Expensive:** Random Forest can be computationally costly, particularly when handling complicated datasets or a big number of Random Forests. It can require a lot of time and resources to train and predict using a large number of Random Forests.
3. **Hyperparameter Tuning:** A number of Random Forest's hyperparameters, including the minimum number of samples per leaf, maximum depth, and number of Random Forests, need to be adjusted. It can take some time to find the ideal set of hyperparameters.

Support Vector Machine:

Strengths:

1. **Effective in High-Dimensional Spaces:** Support vector machines (SVM) are well-suited for datasets with a high number of features due to their performance in high-dimensional spaces.
2. **Robust to Overfitting:** To manage the trade-off between maximizing the margin and minimizing the classification error, SVM employs a regularization parameter (C). This enhances generalization and reduces the risk of overfitting.
3. **Versatility:** By utilizing various kernel functions, such as linear, polynomial, and radial basis function (RBF) kernels, SVM can handle both linear and non-linear classification tasks.
4. **Effective with Small Datasets:** Because SVM concentrates on identifying the ideal decision boundary rather than using the complete dataset, it can function well even with small datasets.

Weaknesses:

1. **Expensive Computation:** SVM can be costly to compute, particularly when working with big datasets. As the quantity of samples and features rises, so does the training time.
2. **Sensitivity to Noise:** Because outliers can dramatically alter the decision boundary's location, SVM is sensitive to noisy data.
3. **Difficult to Interpret:** SVM models can be challenging to understand, particularly when non-linear kernel functions are involved. It can be difficult to understand how the decision boundary and the input features relate to one another.
4. **Parameter Sensitivity:** The kernel parameters and the regularization parameter (C) are two examples of the hyperparameters in SVM. It can be difficult to choose the best values for these parameters, and it might take a lot of tweaking.

To sum up, Random Forest offers feature importance measures, is resilient, and manages missing values effectively. It is less interpretable and can be computationally expensive. On the other hand, SVM is flexible, strong against overfitting, and efficient in high-dimensional spaces. Nevertheless, it can be costly to compute, noisy, and challenging to understand. The particulars of the task, the size and makeup of the dataset, and the trade-offs between interpretability and computational efficiency all play a role in which Random Forest or SVM is selected.

Potential Improvements or Optimizations for Future Work

The following are some possible enhancements or optimizations for upcoming work based on the information provided:

1. **Feature Engineering:** To extract more significant information from the data, it would be advantageous to investigate various feature engineering techniques. To capture intricate relationships, this may entail adding new features, altering current features, or merging several features.

2. **Model Selection:** Examine various ensemble techniques or machine learning algorithms to determine whether they can outperform the Support Vector Machine (SVM) model employed in the current work. Trying several different algorithms and comparing their results to select the best one is always a good idea.
3. **Hyperparameter Tuning:** While GridSearchCV was used for the hyperparameter tuning process, alternative hyperparameter combinations might produce better outcomes. To determine the ideal set of hyperparameters, it would be worthwhile to investigate more sophisticated hyperparameter optimization methods like RandomizedSearchCV or Bayesian optimization.
4. **Cross-Validation Strategy:** The current work employed a popular 5-fold cross-validation strategy. It would be worthwhile to look into whether better model performance could be achieved by utilizing a different number of folds or a different cross-validation technique (such as stratified cross-validation).
5. **Data Augmentation:** To create more artificial data points, data augmentation techniques can be used if the dataset is unbalanced or limited. This may enhance the model's capacity for handling class imbalances and generalization.
6. **Ensemble Methods:** To integrate several models and enhance overall performance, investigate ensemble techniques like bagging, boosting, or stacking. When multiple models are combined, ensemble methods can often yield superior outcomes.
7. **Interpretability of the Model:** SVM models are infamous for being difficult to understand. To find out how the model makes decisions and how important various features are, think about utilizing strategies like feature importance analysis or model-agnostic interpretability methods (like SHAP values).
8. **Performance Metrics:** Take into account assessing the model's effectiveness with other metrics, such as area under the receiver operating characteristic curve (AUC-ROC), precision, recall, or F1-score, in addition to accuracy. These metrics offer a more thorough comprehension of the model's operation, particularly when misclassification costs or class imbalances are present.
9. **Data Preprocessing:** Verify that the procedures used for data preprocessing are suitable for the particular issue at hand. This could entail dealing with outliers, missing values, or scaling the features to make sure they are all on the same scale.
10. **Model Deployment:** Take into account the computational efficiency and scalability of the model if it is to be used in a real-world environment. This could entail investigating distributed computing frameworks, applying dimensionality reduction techniques, or optimizing the model's implementation.

These recommendations are meant to enhance the interpretability, generalizability, and overall performance of the model. To determine which combination of strategies works best for the particular issue at hand, it is crucial to experiment and iterate with various approaches.